

PROJECT SCOPE DOCUMENT

LocalLens

AI-Powered Local Discovery Agent

An Individual Engineering Growth Project — AI / LLM / ML Track

Domain	Stack	Format
AI / LLM / ML Agents	Python (100% Free)	Individual Project

Version 1.0 | May 2025 | Engineering Team — AI Growth Track

1. Project Overview

1.1 What Is LocalLens?

LocalLens is a conversational AI agent that takes a plain English question from a user and returns a ranked, well-summarized list of local businesses, services, or organizations — pulling live data from the web, analyzing public reviews and ratings, and presenting results in a clear, human-readable format.

Think of it as a smart assistant that sits between the user and the internet. Instead of the user having to open Google Maps, then Yelp, then read 50 reviews themselves, LocalLens does it all in one shot: it understands what the user wants, figures out where they are, searches across multiple sources, reads the reviews, scores each option, and then explains the top results in plain language.

1.2 The Core Problem It Solves

Today, finding the best local option for anything — a restaurant, a service, a place, a business — requires the user to:

- Type a search into Google and wade through ads and SEO-gamed results
- Cross-reference multiple platforms (Google Maps, Yelp, TripAdvisor) manually
- Read dozens of reviews and try to form an opinion from inconsistent information
- Repeat the process if the first result is closed, too expensive, or irrelevant

LocalLens eliminates all of that. A user types one natural language query and gets back a ranked, reviewed, and summarized list of the best options — accurate, fast, and without the noise.

1.3 How It Works — The Full Journey

Here is the full lifecycle of a single user query, from input to output, so engineers understand exactly what the system does end-to-end before they start building:

Step 1 — User asks a question in plain English

"Find me the best 3 sushi restaurants near me" or "Give me 5 tax filing companies near Irvine CA" or "Best co-working spaces open on weekends in Austin"

Step 2 — Intent Parser understands the query

The system uses an LLM to extract structured meaning from the raw text: What category? How many results? What location? Any filters (price, open now, rating threshold)? It converts the human question into a machine-readable intent object that downstream modules can act on.

Step 3 — Location is resolved to coordinates

"Near me" is resolved via the user's IP address using a free geolocation service. Named places like "Irvine CA" or "downtown Austin" are converted to lat/lng coordinates using Nominatim (OpenStreetMap's free geocoding API). This produces a geographic bounding box to search within.

Step 4 — The Search Agent finds matching places

The agent queries free public data sources — Overpass API (OpenStreetMap data), DuckDuckGo search, and direct web scraping via Playwright — to find businesses or places matching the category within the target area. Multiple sources are tried in sequence with fallback logic.

Step 5 — Reviews and ratings are aggregated

For each result, the system scrapes publicly available ratings, review counts, and review text from accessible web pages. A local HuggingFace sentiment model reads the review text and extracts signals: overall tone, recurring themes (quality, service, price, atmosphere), and recency patterns.

Step 6 — Results are scored and ranked

A configurable scoring engine combines multiple signals — star rating, number of reviews, sentiment score, and recency of reviews — into a single composite score per result. The top N results (as requested by the user) are selected and ordered.

Step 7 — An LLM writes a natural language summary

For each top result, a local LLM generates a 2-3 sentence human-readable summary based strictly on the retrieved data. The LLM is given the scraped facts as context and is explicitly instructed not to add anything not found in that data — preventing hallucination.

Step 8 — The final answer is returned to the user

The user receives a clean, ranked list: business name, address, composite score, hours, and a natural language summary explaining why it is a good pick. All of this from one plain English question, in under 10 seconds.

1.4 Sample Queries the System Must Handle

User Query	Location Type	Category
Find me best 3 sushi restaurants near me	Geolocation / IP	Restaurant
Best 5 meditation centers near me	Geolocation	Wellness / Spiritual
5 tax filing companies in Irvine CA	Named city	Financial Services
Top 4 grocery stores near zip code 10001	Zip code	Retail
Best co-working spaces open weekends in Austin	Named city + filter	Workspace
3 highly rated dentists within 10km of me	Geolocation + radius	Healthcare
Affordable yoga studios near downtown Seattle	Named area + filter	Fitness

1.5 Why This Project for Engineer Growth?

This project is intentionally designed as a full-stack AI challenge. It is not a tutorial and not a hello-world LLM app. Engineers who build this will encounter and solve real problems that exist in production AI systems today:

- How do you make an LLM reliably extract structured data from a messy, unpredictable user sentence?
- How do you build an agent that calls multiple tools in sequence, with error handling and fallback logic?
- How do you crawl and normalize data from sources that have no clean API?
- How do you design a scoring system that feels accurate and trustworthy to a real user?
- How do you prevent an LLM from making things up when summarizing retrieved facts?
- How do you build a pipeline that is fast, observable, and testable end-to-end?

Each of these questions maps directly to a real AI/LLM/ML engineering skill. By the end of this project, the engineer will have built a working agentic system from scratch and will understand every layer of the stack deeply.

2. System Architecture

LocalLens is built as a multi-agent pipeline. Each layer is a distinct engineering responsibility, allowing engineers to own a vertical slice from prompt to output. The system is modular — each module can be built, tested, and run independently before being connected to the full pipeline.

2.1 High-Level Agent Pipeline

#	Layer	Responsibility	Output
1	Intent Parser Agent	Extract: query type, count, category, location, filters	Structured JSON intent
2	Location Resolver	Resolve 'near me', city names, zip codes to lat/lng	Geo coordinates
3	Search & Crawl Agent	Query Overpass API, DuckDuckGo, web scraping fallback	Raw business listings
4	Review Aggregator	Scrape ratings and reviews from public web sources	Enriched data per listing
5	Ranking & Scoring Engine	Score by rating, review volume, sentiment, recency	Ranked list of N results
6	Summary Generator (LLM)	Generate grounded natural language summary per result	Human-readable output
7	Response Formatter	Format final answer as JSON or chat response	Final response to user

2.2 Data Flow

End-to-End Pipeline
User Query → [Intent Parser] → [Location Resolver] → [Search Agent: Overpass API + DuckDuckGo + Playwright] → [Review Aggregator: public web scraping + HuggingFace NLP] → [Scoring Engine] → [LLM Summarizer (local Ollama)] → [Formatted Output to User]

2.3 Key Design Decisions

- Agent-based architecture: each layer is independently testable and replaceable without breaking other modules

- LLM is used for intent parsing AND final summarization — not for factual data retrieval (prevents hallucination at the data layer)
- Live data only: no static database; crawl on every query with local disk caching to avoid redundant calls
- Fallback chain: Overpass API (OpenStreetMap) → DuckDuckGo search → direct Playwright web scrape
- Confidence scoring: the system flags low-confidence or thin-data results explicitly in the output
- Zero cost to run: every tool and API used is free and open-source — no credit card required

3. Technology Stack (100% Free & Open Source)

Every tool in this stack is completely free. Engineers do not need to spend any money, enter any credit card, or create any paid accounts to build and run this project end-to-end. All services are either open-source libraries, fully public APIs, or tools with no-cost self-hosted options.

Category	Technology	Purpose	Cost
Language	Python 3.11+	Primary language for all modules	Free
LLM Framework	LangChain / LangGraph	Agent orchestration, tool calling, memory	Free (OSS)
LLM — Local	Ollama + Llama 3 / Mistral 7B	Run LLM locally on your machine, no API key needed	Free
LLM — Hosted Alt.	Groq API (free tier)	Fast hosted inference, free tier, no credit card	Free tier
Places / Search	Overpass API (OpenStreetMap)	Business search by category and bounding box	Free
Geocoding	Nominatim (OpenStreetMap)	Convert city names and zip codes to lat/lng	Free
IP Geolocation	ip-api.com	Resolve 'near me' from user IP — no key needed	Free
Web Search	DuckDuckGo (duckduckgo-search)	Fallback search for businesses not in OSM	Free
Web Scraping	Playwright + BeautifulSoup	Crawl public review pages and business listings	Free (OSS)
NLP / Sentiment	HuggingFace Transformers	Local sentiment model for review analysis	Free (OSS)
Vector Store	ChromaDB (local)	Local vector store for embedding and semantic search	Free (OSS)

Category	Technology	Purpose	Cost
API Layer	FastAPI	REST API exposing the agent as a service	Free (OSS)
Caching	diskcache / SQLite	Local query result caching — no Redis or server needed	Free
Observability	Langfuse (self-hosted / free cloud)	Trace and debug LLM + agent steps	Free (OSS)
Testing	pytest	Unit and integration tests for every module	Free (OSS)
Notebooks	Jupyter Notebook	Exploration, scoring evals, prototype testing	Free (OSS)

Zero Cost Setup

The recommended default is Ollama running Llama 3 8B locally — no internet needed after the model download. For engineers without a GPU or who want faster inference, Groq offers a completely free API tier (no credit card) with Llama 3 and Mixtral available. All data sources — Overpass API, Nominatim, ip-api.com, DuckDuckGo — are free with no registration or API keys required.

4. Individual Engineering Modules

Each module below is a self-contained unit of work that one engineer can own end-to-end. Modules are designed for individual growth — each one teaches a different AI/ML/LLM engineering skill set and can be built and tested in isolation before being wired into the full pipeline.

MODULE A — Intent Parser

A. Intent Parser & NLP Engine

Skills: Prompt Engineering, Structured Output, Entity Extraction, LangChain

- Design an LLM prompt to extract structured fields from a raw user query: category, count, location string, distance radius, sort preference, and any filters
- Output a strongly-typed Pydantic schema: { category, count, location, radius_km, sort_by, filters[] }
- Handle ambiguous queries: 'best sushi near me' vs 'top 3 affordable sushi in Seattle open now'
- Use chain-of-thought prompting for complex multi-condition queries with multiple filters
- Build a test suite with 30+ diverse query examples covering different categories and location types

Sample Output Schema

```
{ "category": "sushi restaurant", "count": 3, "location": "near_me", "radius_km": 5, "sort_by": "rating",  
  "filters": ["open_now"] }
```

MODULE B — Location Resolver

B. Location Resolution Service

Skills: Geocoding APIs, Geospatial Logic, Python async, Edge Case Handling

- Resolve 'near me' via IP geolocation using the free ip-api.com service — no API key required
- Parse named locations — 'Irvine CA', 'downtown Austin', 'zip code 10001' — using Nominatim (OpenStreetMap)
- Convert resolved location to a lat/lng bounding box suitable for Overpass API queries
- Handle edge cases: vague area names ('Silicon Valley'), neighborhood names, and landmark references
- Cache resolved locations locally using diskcache to avoid repeat geocoding calls for the same input

MODULE C — Search & Data Agent

C. Multi-Source Search & Crawl Agent

Skills: API Integration, Web Scraping, LangChain Tools, Async Python

- Implement LangChain tool: Overpass API query using lat/lng bounding box and OSM amenity/shop tags for category matching
- DuckDuckGo search as secondary source for businesses and services not well-covered in OpenStreetMap
- Playwright-based scraper as final fallback for business pages discovered via web search
- Normalize all results from different sources into a single unified BusinessListing schema
- Implement retry logic, per-source rate limiting, and graceful degradation when a source is unavailable

MODULE D — Review Aggregator

D. Review & Rating Aggregation Engine

Skills: Web Scraping, NLP, Sentiment Analysis, HuggingFace Transformers

- Scrape publicly accessible rating data and review text for each business result using Playwright and BeautifulSoup
- Run sentiment analysis using a local HuggingFace model (e.g. distilbert-base-uncased-finetuned-sst-2-english)
- Compute derived signals per business: positive review percentage, negative percentage, and top recurring themes
- Apply recency weighting: reviews from the last 6 months are weighted more heavily than older ones
- Flag low-confidence data: results with fewer than 5 reviews, no rating, or very sparse review text

MODULE E — Scoring & Ranking Engine

E. Intelligent Ranking & Scoring Engine

Skills: Scoring Algorithms, Feature Engineering, Configurable Pipelines

- Design a composite score formula: 40% star rating + 30% review count + 20% sentiment score + 10% recency signal
- Normalize scores from different sources to a common 0-100 scale for fair cross-source comparison
- Support category-specific weight overrides via config (e.g. price sensitivity weighted higher for 'affordable' queries)
- All scoring weights live in a YAML config file — no hard-coded values anywhere in the scoring code
- Write evaluation notebooks comparing scoring formulas against manually labeled expected results

MODULE F — LLM Summarizer

F. Conversational Summary Generator

Skills: Advanced Prompt Engineering, Grounding / RAG Pattern, Hallucination Prevention

- Generate a 2-3 sentence natural language summary for each ranked result using a local Ollama model
- Summaries must highlight: what makes this place stand out, who it suits best, and one specific detail from the reviews
- Inject scraped review text as grounding context — the LLM is explicitly constrained to only use facts from this context
- Implement a basic hallucination check: verify that specific claims in the summary exist in the source data

- Format the final response output: ranked list with name, address, composite score, hours, and the generated summary

MODULE G — API & Interface Layer

G. FastAPI Service & Interface

Skills: FastAPI, REST API Design, Async Python, Local Deployment

- Build a POST /search endpoint accepting: { query: string, user_ip?: string } and returning ranked results
- Implement streaming response via Server-Sent Events (SSE) so results appear progressively as each step completes
- Add /health and /metrics endpoints with structured JSON logging for observability
- Local disk-based caching layer to skip repeat crawls for identical queries within a configurable TTL window
- Optional stretch: build a simple HTML + JavaScript chat interface that calls the API and streams results to the browser

5. Delivery Milestones

Week	Milestone	Deliverable	Review
Week 1-2	Foundation	Module A: Intent Parser working with 30+ test queries	Demo + code review
Week 3	Data Layer	Module B + C: location resolved, OSM search returning raw results	API response demo
Week 4	Intelligence	Module D + E: reviews scraped, sentiment run, rankings computed	Score comparison review
Week 5	LLM Layer	Module F: grounded local LLM summaries per result	Quality eval session
Week 6	Integration	Module G: full FastAPI service running end-to-end	End-to-end demo
Week 7-8	Polish	Edge cases, evals, documentation, optional UI, retrospective	Final showcase

6. Success Criteria & Evaluation

6.1 Functional Requirements

- System handles 20+ diverse query types across at least 5 different business and service categories
- Returns accurate results (manually verified spot-check) for at least 85% of test queries
- Correctly resolves location for all four input types: 'near me', city name, zip code, and named neighborhood
- Full pipeline completes in under 10 seconds per query on a standard laptop without GPU
- Summaries contain no hallucinated facts — every claim in the output must exist in the retrieved source data

6.2 Technical Excellence Criteria

- All modules have unit tests with at least 70% code coverage via pytest
- All agent steps are traced in Langfuse — no silent or untracked LLM calls in production flow
- Local caching layer reduces redundant web calls by more than 60% when the same query is repeated
- Scoring weights are documented, externalized to YAML, and adjustable without changing any code
- README includes: environment setup steps, architecture overview, and at least 5 example inputs with expected outputs

6.3 Engineer Growth — What You Will Be Able to Do

Skill Area	What You Will Be Able to Do After This Project
LLM / Prompt Engineering	Design reliable structured output prompts, chain-of-thought reasoning, and grounded summarization that prevents hallucination
Agentic AI Systems	Build multi-step autonomous agents with tool calling, fallback chains, and error recovery using LangGraph
NLP & Sentiment Analysis	Apply local HuggingFace transformer models for real-world review classification and feature extraction
Data Crawling & Integration	Integrate multiple public APIs and write robust scrapers with source normalization and deduplication logic
Scoring & Ranking	Design and tune a composite scoring algorithm with configurable, externalized, and testable weights
Production Python	Build async pipelines, local caching, observability tracing, and a deployable FastAPI service from scratch

7. Stretch Goals & Advanced Extensions

For engineers who complete the core modules, these extensions push into advanced AI and production engineering territory. All stretch goals use only free tools and require no additional cost.

7.1 Advanced AI Extensions

- Conversational memory: support follow-up queries like 'show me cheaper options' or 'only ones open right now' — agent retains prior context within a session
- Voice input: accept spoken queries using OpenAI Whisper (local, free) for transcription before passing to the intent parser
- Fine-tuned intent classifier: collect the test queries from Module A and train a small BERT-based classifier instead of relying on LLM prompting at inference time
- Clarifying questions: when the intent parser detects low-confidence extraction, the agent asks one targeted clarification before proceeding with the search

7.2 Infrastructure Extensions

- Async parallel crawling: run the search agent and review aggregator concurrently with asyncio to cut total query latency in half
- Dockerize the full stack: produce a single docker-compose.yml that spins up the API, Ollama model server, and Langfuse together
- Semantic business search: embed business descriptions using a free sentence-transformer model and enable semantic similarity ranking as an alternative to keyword matching
- Simple frontend: a lightweight HTML and JavaScript chat interface that streams results from the FastAPI SSE endpoint in real time

7.3 Data Quality Extensions

- Low-data detection: when a business has fewer than 5 reviews or no rating, the system clearly warns the user that this result has limited data
- Duplicate merging: detect and merge listings of the same business that appear from multiple sources (Overpass + web scrape) using name and coordinate proximity matching
- Category auto-tagging: use zero-shot LLM classification to assign OSM-compatible category tags to businesses discovered via web search that lack structured tags

— End of Document —